Team Name : Bk

Team Leader Name : Bhoomi Kaushik (Meghna Kaushik) — mkgapbvk@gmail.com

Problem Statement : Rank 100,000 candidates for Redrob's Senior AI Engineer (Founding Team) role under a hidden NDCG/MAP/P@10 formula, with a built-in anti-keyword trap and ~80 honeypot profiles.

Solution Overview

- **What is the proposed solution?**

A 4-stage funnel — BM25 sparse retrieval → TF-IDF/SVD dense filter → rule-based feature scoring + honeypot detection → learned reranker ensemble (LightGBM LambdaMART, sklearn GBR fallback) — producing a ranked top-100 with grounded, template-based reasoning per candidate.

Rule score is the auditable backbone (every number traces to a named field); the learned model trains on our own heuristic pseudo-labels to catch nonlinear interactions the fixed weight formula can't express.

What differentiates this from traditional candidate matching?

We verified the JD's stated anti-keyword trap against the data before writing scoring code: buzzwords (Pinecone, RAG, Embeddings) appear in ~5,000 candidates (~5%) spread almost uniformly across irrelevant titles, while real signal (BM25, Learning to Rank, OpenSearch) is held by 97.7% of the 750 true ML/AI-titled candidates vs. only 13.8% of everyone else.

Honeypot detection is a multiplicative gate, not an additive feature — disabling it in our own ablation sends NDCG@10 to 0.0 because every top-10-by-rule-score candidate becomes a honeypot.

JD Understanding & Candidate Evaluation

- **Key requirements extracted from the JD**

Title tiers from exact match (Senior AI Engineer, ML Engineer, NLP Engineer…) down to explicitly penalized titles (AI Research Engineer, Junior ML Engineer, Computer Vision Engineer).

Required skills (BM25, Weaviate/Qdrant/Milvus/OpenSearch, Learning to Rank, Python/PyTorch), 6–8y experience with applied ML at a product company, Pune/Noida or willing to relocate.

Explicit disqualifiers: service-only career history, LangChain without pre-LLM evidence, title-chasing (3+ employers, <18mo avg tenure), CV/speech-only without NLP/IR.

Which signals matter most for ranking

Semantic similarity to JD text (30%), category-weighted skill match (20%), experience/title fit (15%), trust/honeypot status (10% + a multiplicative gate), behavioral availability — recruiter response rate, notice period, last active date (10% + 5%).

Ranking Methodology

- **How the system retrieves, scores, and ranks**

Stage 1: BM25Okapi sparse retrieval, 100,000 → top 1,000.

Stage 2: TF-IDF + SVD (128 components, fit once on the full corpus) dense filter, 1,000 → top 300.

Stage 3: full feature scoring (skill/title/company/career/behavior/trust) + honeypot probability for the 300 survivors.

Stage 4: 5-fold cross-validated GBT reranker on heuristic pseudo-labels; final_score = 0.5×rule_score + 0.5×model_score.

Models, algorithms, and why

BM25Okapi (rank-bm25) for cheap lexical filtering before anything expensive runs.

TF-IDF+SVD instead of sentence-transformers — CPU-only/no-network constraint, ~22s for the full 100K corpus, fully interpretable.

LightGBM LambdaMART intended; sklearn GradientBoostingRegressor fallback is what actually ran in our dev sandbox (lightgbm wasn't importable there).

Explainability & Data Validation

- **How ranking decisions are explained**

src/reasoning.py builds a 4-clause string per candidate (experience, evidence/matched-skills + role snippet, behavior, concerns) entirely from computed fields — no LLM call at inference time.

How we prevent hallucinated or unsupported justifications

Every clause traces to a specific field (matched_required_skills, honeypot_probability, negative_reasons, title_penalty_note) — nothing is free-generated text.

Data validation

Honeypot thresholds counted directly from the data: 410 found at full 100K scale vs. the spec's stated ~80 — a known over-triggering gap in one rule, reported openly rather than tuned away (see challenge_analysis.md).

validate_submission.py enforces the exact output schema — header, row count, candidate_id pattern, rank uniqueness, score monotonicity — before a run counts as done.

End-to-End Workflow

- **Complete workflow, JD input to ranked output**

1. Load candidates.jsonl (100,000 rows, 0 malformed in the real release).
2. BM25 filter to 1,000.
3. Fit TF-IDF/SVD on the full corpus, dense-filter to 300.
4. Score honeypot probability + full feature dict for the 300 survivors.
5. Generate heuristic pseudo-labels (0–4 relevance tiers).
6. 5-fold CV diagnostics + train GBT reranker on the full 300.
7. Ensemble score, sort, take top 100, generate grounded reasoning.
8. Write submission.csv + metrics_report_data.json.

One command: python rank.py --candidates candidates.jsonl --out submission.csv — 56.7s wall clock against the real 100,000-row file, well under the 5-minute budget.

System Architecture

rank.py (orchestrator) calls into:

src/ingest.py — candidate loading, survives 1 malformed record

src/semantic.py + src/retrieval.py — BM25 + TF-IDF/SVD funnel

src/honeypot.py — rule-based fake-profile detection

src/features.py — skill/title/company/behavior/trust scoring

src/pseudo_labels.py — weak-supervision relevance tiers

src/train_ranker.py — LightGBM/sklearn-GBR reranker + cross-validation

src/reasoning.py — grounded, template-based explanations

src/metrics.py — NDCG/MAP/P@k, own implementation, unit-tested

weighted_job_representation.json is the structured JD every module above reads from.

Results & Performance

- **Cross-validated against our own pseudo-labels (no real ground truth exists)**

5-fold, candidate-level: NDCG@10 avg ~0.997, MAP avg 1.00, P@10 avg 0.86 (two folds at 0.7 — noisy on ~60-candidate folds, not a pipeline issue).

Headline ablation result

Disabling honeypot detection sends NDCG@10 from 0.979 to 0.0 — every honeypot that reaches the rule-score top 100 stays there unfiltered; with detection on, 0 do.

Honest negative result, reported as found

Flat vs. category skill weighting is mixed at full 100K scale (unlike our dev-scale run, where flat won cleanly): category wins NDCG@10 (0.828 vs 0.822), flat wins NDCG@50/MAP — we report the split rather than the cleaner-looking dev-scale number (see metrics_report.md). The qualitative case for category weighting still holds from our baseline test.

Technologies Used

- rank-bm25 (BM25Okapi) — sparse retrieval, fast/dependable baseline, no learned weights.

scikit-learn (TfidfVectorizer + TruncatedSVD, GradientBoostingRegressor, KFold) — dense retrieval substitute for sentence-transformers and the reranker fallback.

LightGBM (LGBMRanker, lambdarank objective) — intended reranker backend; not importable in our dev sandbox, sklearn GBR is what we actually validated against.

pandas / numpy — feature table and scoring.

python-docx — extracting JD text from the provided .docx.

Chosen for: CPU-only, no-network, ≤16GB RAM, ≤5 min wall clock — every dependency had to survive a clean-room run with none of our dev assumptions.

Submission Assets

GitHub repo: https://github.com/bhoomiihere/rankerIQ

Contents: rank.py, src/, challenge_analysis.md, metrics_report.md, experiments/exp_log.md, README.md, Dockerfile, requirements.txt, submission.csv, tests/.

Reproduce: pip install -r requirements.txt && python rank.py --candidates candidates.jsonl --out submission.csv && python validate_submission.py submission.csv

Sandbox/demo link: not yet deployed — see README “Deploying the sandbox link” for the planned Streamlit/HF Spaces approach.

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU